

Comparative Analysis of **sochiautoparts** Open-Source Projects

Deep security, paywall, and functionality analysis of gitmoji-ai, devbadge, and repokit repositories. Examining the gap between advertised Pro features and actual implementation.

May 2026 · Independent Analysis
gitmoji-ai · devbadge · repokit

PAYWALL ANALYSIS

FEATURE VERIFICATION

SECURITY AUDIT

Table of Contents

1. Executive Summary	3
2. Common Patterns Across All Three Projects	3
2.1 StarsPay Payment System	3
2.2 Paywall Architecture	4
2.3 Advertised vs. Real Features Pattern	4
3. gitmoji-ai: Detailed Analysis	4
3.1 Project Overview	4
3.2 Paywall Bypass Methods	4
3.3 Advertised vs. Real Features	5
3.4 Critical Bugs	6
4. devbadge: Detailed Analysis	6
4.1 Project Overview	6
4.2 Paywall Bypass Methods	6
4.3 Advertised vs. Real Features	7
4.4 Critical Bugs	7
5. repokit: Detailed Analysis	8
5.1 Project Overview	8
5.2 Paywall Bypass Methods	8
5.3 Advertised vs. Real Features	8
5.4 Critical Bugs	9
6. Security Vulnerability Comparison	10
6.1 Critical Vulnerabilities	10
6.2 Paywall Effectiveness Assessment	11
7. Feature Delivery Assessment	11
7.1 Pro Feature Delivery Rate	11
7.2 Free Tier Value	11

8. Code Quality Comparison	12
9. Overall Assessment and Scoring	12
10. Recommendations	13
10.1 For Potential Users	13
10.2 For the Author	13
10.3 For GitHub	14

1. Executive Summary

This report presents a comprehensive security and functionality analysis of three open-source repositories published under the GitHub account **sochiautoparts**: **gitmoji-ai**, **devbadge**, and **repokit**. All three projects share a common architecture pattern: a free tier with limited features, a Pro tier gated by a license key, and a Telegram-based payment system called StarsPay. Our analysis reveals that all three repositories suffer from trivially bypassable paywalls, advertised Pro features that do not exist in the codebase, and critical bugs that undermine core functionality.

The overarching pattern across all three projects is consistent: the paywall exists only as a superficial gate that can be circumvented with minimal effort (setting an environment variable, creating a cache file, or directly calling internal functions). More concerning, many Pro features advertised in the README are either completely unimplemented, non-functional on the target platform (GitHub README), or implemented as placeholder stubs. The payment system relies on a Telegram bot with no transparency, no dispute mechanism, and stores license data in a public GitHub repository. Users who pay for Pro features are, in most cases, paying for functionality that does not exist.

Metric	gitmoji-ai	devbadge	repokit
Language	Python	Python	Node.js
Paywall Bypass	Trivial	Trivial	Trivial
Pro Features Real	2 of 7	2 of 7	5 of 30
Critical Bugs	3	4	5
Hardcoded Secrets	OAuth Client ID	None	API Key
GitHub Action	Partially broken	Broken	Non-functional
Overall Score	4/10	5/10	3/10

Table 1. Comparative overview of the three repositories

2. Common Patterns Across All Three Projects

2.1 StarsPay Payment System

All three repositories use the same payment infrastructure: StarsPay, a Telegram bot (**@allstarspay_bot**) that processes payments in Telegram Stars and generates license keys. The integration model is identical across projects: the user pays via the bot, receives a key in format SP-XXX-XXXX-XXXX (where XXX identifies the product: GMA for gitmoji-ai, DVB for devbadge, RPK for repokit), and then activates the key via a CLI command. The verification process fetches a public JSON file from the **sochiautoparts/stars-pay-bot** GitHub repository, which contains all license hashes. This public exposure of license data is a fundamental security weakness.

The pricing model is also consistent: approximately 149 Stars per month, 999 Stars per year, and 2999 Stars for a lifetime license. Telegram Stars is a closed ecosystem currency with no external audit trail, no chargeback mechanism, and no regulatory oversight. Users who purchase Pro licenses have no recourse if the features advertised do not materialize. The StarsPay API itself runs on Render free tier, which spins down after inactivity, causing frequent offline fallback to the locally cached license data, which can be trivially forged.

2.2 Paywall Architecture

Every project implements a three-tier license verification system: (1) a local cache file with a time-to-live window, (2) a remote check against the public licenses.json on GitHub, and (3) an API fallback to the StarsPay server. All three tiers are vulnerable. The local cache can be manually created or edited to grant Pro status. The public GitHub JSON exposes all license hashes for brute-force attacks (SHA-256 truncated to only 16 hex characters, i.e. 64 bits of security). The API fallback silently fails on network errors, defaulting to the cache, which is forgeable. Additionally, the environment variable check in gitmoji-ai (**settings.is_pro**) simply checks if the string is non-empty, making any non-empty value a valid Pro credential.

2.3 Advertised vs. Real Features Pattern

A consistent pattern emerges: README files prominently advertise Pro features that are either unimplemented, partially implemented as stubs, or non-functional on the target platform. In gitmoji-ai, custom commit styles and team features are marked as "Coming Soon" but have zero code. In devbadge, CSS animations are advertised for GitHub profiles where GitHub strips all <style> tags from SVGs. In repokit, 25 of 30 templates are placeholder README files with no actual code generation. This pattern suggests a deliberate strategy of advertising future or non-existent features to drive Pro license purchases.

3. gitmoji-ai: Detailed Analysis

3.1 Project Overview

gitmoji-ai is a CLI tool that generates AI-powered commit messages using OpenAI GPT models. It supports conventional commits, emoji-style commits, and plain text commits. The tool integrates with git hooks, provides a changelog generator, and offers a GitHub Action for automated commit message generation in CI/CD pipelines. The free tier provides 50 AI commit messages per month and 3 changelog generations per month, while the Pro tier promises unlimited usage and additional features. The project is written in Python with Typer as the CLI framework, Rich for terminal output, and httpx for HTTP requests. All declared dependencies are genuinely used, which is a positive quality indicator compared to the other two projects.

3.2 Paywall Bypass Methods

Method	Difficulty	Details
Set GMAI_PRO_LICENSE_KEY=x	Trivial	settings.is_pro returns True for any non-empty string; short-circuits real verification
Edit ~/.gitmoji-ai/usage.db	Trivial	Insert row into license table with future expires_at and last_verified_at
Patch _pro_cache dict	Easy	Set _pro_cache["result"] = True in Python memory
Brute-force key_hash	Moderate	16 hex chars = 64 bits; public JSON reveals all targets

Table 2. gitmoji-ai payroll bypass methods

The most critical flaw is that the watermark removal check (**cli.py:183-185**) uses the weak **settings.is_pro** property rather than the real verification function **usage.is_pro()**. Furthermore, the **check_limit()** function at **usage.py:109** short-circuits with **if settings.is_pro or is_pro()**, meaning the real verification is never called when GMAI_PRO_LICENSE_KEY is set to any string. Additionally, the environment variable names are inconsistent: GMAI_PRO_LICENSE_KEY (**config.py:25**) versus LICENSE_KEY (**usage.py:266, 355**), causing confusion about which variable to set.

3.3 Advertised vs. Real Features

Advertised Feature	Tier	Reality
AI commit messages (50/mo free, unlimited Pro)	Free/Pro	Real, but limit trivially bypassed
AI changelog (3/mo free, unlimited Pro)	Free/Pro	Real, but limit trivially bypassed
7 languages (EN, RU, ES, DE, FR, JA, ZH)	Free/Pro	Only EN and RU have system prompts; 5 are fake
No watermark	Pro	Gate is trivially bypassed via settings.is_pro
Custom commit styles	Pro	NOT IMPLEMENTED - zero code exists
Team features	Pro	NOT IMPLEMENTED - zero code exists
Priority support	Pro	NOT VERIFIABLE - no mechanism exists

Table 3. gitmoji-ai feature comparison: advertised vs. reality

The language support claim is particularly misleading. While the README advertises 7 languages, only English and Russian have actual system prompts in the code (**ai_engine.py:142** and **:169**). The other 5 languages rely on appending a "Language: es" tag to the English system prompt and hoping GPT figures out the translation. This is not internationalization; it is a hope-based approach that produces inconsistent results. The changelog command only accepts "en" and "ru" for the --lang flag (**cli.py:202**), further confirming that only 2 languages are truly supported.

3.4 Critical Bugs

Bug 1: Two competing GitHub Action files. The root `action.yml` uses `"gmai suggest --quiet"` (non-destructive, just suggests), while `action/action.yml` uses `"gmai commit --yes"` which actually creates a commit in CI. This is dangerous because auto-committing in CI can cause infinite loops, override human review, and create unexpected repository state. Neither action passes the Pro license key, so both always run in free tier mode with watermarks.

Bug 2: suggest command bypasses rate limiting. The `suggest` command does not call `check_limit()`, meaning users can call `"gmai suggest --quiet"` unlimited times regardless of Pro status. This completely undermines the free tier usage limits, as users can use the suggest output to manually commit without consuming any quota. The suggest command also does not add the "Generated by gitmoji-ai (free)" watermark, giving free users a watermark-free experience through the back door.

Bug 3: device_flow_login() is a stub. The function at `sponsors.py:177-204` is named `device_flow_login()` but simply prints instructions and returns `None`. The `HTTPServer` and `BaseHTTPRequestHandler` imports at `sponsors.py:16` and the `webbrowser` import at `sponsors.py:15` are leftover artifacts from an unimplemented OAuth device flow. GitHub PATs are stored in plaintext at `~/.gitmoji-ai/auth.json` with no file permission restrictions (default 644 = world-readable).

4. devbadge: Detailed Analysis

4.1 Project Overview

`devbadge` generates SVG badges for GitHub profiles, README files, and documentation. It offers 5 free badge types (commits, languages, stats, activity, profile) and 3 Pro badge types (coffee, spotify, weather), along with 6 free themes and 2 Pro themes (neon, aurora). The tool fetches data from the GitHub API (REST and GraphQL) and generates static SVG files. Unlike competing services like Shields.io or GitHub README Stats, `devbadge` generates local SVG files rather than dynamic URL-based badges, which limits their use for always-up-to-date profile displays.

4.2 Paywall Bypass Methods

Method	Difficulty	Details
Create <code>~/.devbadge/license_cache.json</code>	Trivial	Write JSON with <code>valid=true</code> , <code>expires_at</code> far future
Direct Python API call	Easy	<code>CoffeeBadge.generate(is_pro_user=True)</code> bypasses all checks
Offline fallback	Easy	Block <code>starspay-api.onrender.com</code> ; forces cache-only check
Brute-force <code>key_hash</code>	Moderate	SHA-256 truncated to 16 hex; public JSON exposes targets

Table 4. `devbadge` paywall bypass methods

A critical implementation flaw exists in the cache verification at `config.py:118`. The code attempts to compare a Unix timestamp (`time.time()`) against an ISO 8601 string stored in the cache, resulting in a `TypeError` (float minus string). This means the 24-hour re-verification check never works correctly. The cache is either always considered expired (if the `TypeError` is caught by the outer try block) or always trusted (if the fallback path is taken). Either way, the cache mechanism is fundamentally broken, further weakening the paywall.

4.3 Advertised vs. Real Features

Advertised Feature	Tier	Reality
3 Pro badges (coffee, spotify, weather)	Pro	Coffee works (trivial); Spotify/Weather are placeholders with no API
2 Pro themes (neon, aurora)	Pro	Colors defined but <code>animated=True</code> flag is never consumed by rendering code
4 animations (pulse, gradient, typing, sparkle)	Pro	All use <code><style></code> tags stripped by GitHub; module docstring admits this
No watermark	Pro	Works correctly
Custom colors	Pro	FAKE: <code>custom_colors</code> field exists in config but is never read by any badge code
Priority support	Pro	NOT VERIFIABLE: no support system exists

Table 5. devbadge feature comparison: advertised vs. reality

The animations feature deserves special scrutiny. The `animations.py` module docstring (lines 3-5) explicitly states: "GitHub README sanitizes SVG and strips `<style>` tags, making CSS animations non-functional on GitHub profiles." Despite this admission in the code, the README markets animations as a Pro feature without disclosing this critical limitation. The pulse animation claims to use "inline style attributes that survive sanitization," but the actual implementation (lines 27-36) generates a `<style>` block with `@keyframes` and CSS classes, which is exactly the approach that GitHub strips. Only gradient and sparkle animations work in a standalone browser, but the primary use case for profile badges is GitHub README, where none of them function.

4.4 Critical Bugs

Bug 1: GitHub Action is broken. The `action.yml` at line 37 passes `--badges` but the CLI argument is `--badge` (singular, `cli.py:36`). This argument mismatch causes the Action to fail with an unrecognized argument error every time. No user can successfully use the GitHub Action as shipped.

Bug 2: Cache timestamp type mismatch. The `config.py:118` code does `time.time() - cache_time` where `cache_time` is an ISO 8601 string, not a Unix timestamp. This causes a `TypeError` that is silently caught, making the 24-hour cache re-verification mechanism non-functional.

Bug 3: GraphQL commit count is annual, not lifetime. The `fetch_commit_count_graphql()` function uses `contributionsCollection` which defaults to the current year only unless date ranges are specified. The function name says "total" but it returns only the current year's commits, producing misleading badges for users with longer contribution histories.

Bug 4: Emoji in SVG does not render. The stats badge (`badges.py:309-312`) and coffee badge (`badges.py:381`) use emoji characters in SVG `<text>` elements, which do not reliably render in SVG viewers or GitHub. These appear as blank squares or missing glyphs, undermining the visual quality of the generated badges.

5. repokit: Detailed Analysis

5.1 Project Overview

repokit is a project scaffolding CLI tool that generates boilerplate code for 30+ technology stacks. It positions itself as a comprehensive project generator with database integration, authentication, Docker configuration, and CI/CD setup. The free tier includes 5 templates (Next.js, Express, React, FastAPI, Vue), while the Pro tier promises 25+ additional templates for frameworks like Django, Spring Boot, Flutter, Svelte, Angular, and more. The tool is written in Node.js with Inquirer for interactive prompts, Handlebars for template processing, and Chalk for terminal colors.

5.2 Paywall Bypass Methods

Method	Difficulty	Details
Create <code>~/.repokit-license</code>	Trivial	Write JSON with <code>valid=true</code> , <code>plan=pro</code> , <code>expires_at=2099</code>
Offline fallback	Trivial	Block <code>starspay-api.onrender.com</code> ; forces local cache only
Hardcoded API key in source	N/A	<code>sk_starspay_repokit_xK9mP2vL4nQ7rW</code> at <code>license.js:9</code>
Brute-force key format	Easy	<code>SP-RPK-[A-Z0-9]{4}-[A-Z0-9]{4}</code> = only 36^8 combinations

Table 6. repokit paywall bypass methods

The most alarming finding in repokit is the hardcoded API key `sk_starspay_repokit_xK9mP2vL4nQ7rW` at `license.js:9`. This key is shipped in the npm package and allows anyone to impersonate the application when communicating with the StarsPay API. An attacker could use this key to query license information, forge verification responses, or potentially abuse the API endpoint. The key format uses a "sk_" prefix (secret key), suggesting it was intended to be kept confidential, yet it is exposed in the open-source codebase. This is a critical security vulnerability that goes beyond the paywall bypass issue.

5.3 Advertised vs. Real Features

The most significant finding in repokit is that **83% of Pro templates (25 out of 30) are placeholder README files with no actual code generation**. Only 5 templates (Next.js, Express, React, FastAPI, Vue) have real Handlebars template files (.hbs) that generate runnable projects. The remaining 25 templates exist as directories containing only a README.md placeholder file. Despite this, the generator code (generator.js) contains elaborate functions for each Pro template, generating package.json files, Docker configurations, and skeleton code structures. However, these functions are **dead code** because of a critical logic flaw.

The bug is at generator.js:71-77. The code checks if a template directory exists and contains any files: "if (fs.existsSync(templateDir) && fs.readdirSync(templateDir).length > 0)". Since ALL Pro template directories contain at least a README.md, this condition is always true, and processTemplateDir is called instead of generateDynamically. The processTemplateDir function only copies .hbs files, and since Pro template directories have no .hbs files (only README.md), the output is a project directory with nothing but a README.md. The 25 elaborate generator functions (lines 987-1925) are never executed. This means even the partial implementations described in those functions (Django settings, Spring Boot controllers, etc.) are never generated for users.

Category	Templates	Status
Working (real .hbs files)	nextjs, express, react, fastapi, vue	Functional
Partial (dynamic generators exist but unreachable)	django, spring-boot, go-api, rust-api, flutter, react-native, electron, discord-bot, telegram-bot, chrome-extension, nextjs-fullstack, nextjs-ecommerce, nextjs-saas	Dead code
Fake (only README.md placeholder)	angular, laravel, rails, graphql, microservices, cli-tool, python-cli, desktop-tauri, wordpress, svelte, remix, nestjs, svelte-fullstack, kotlin-api, vue-fullstack, astro	Placeholder

Table 7. repokit template quality breakdown

5.4 Critical Bugs

Bug 1: Dead code - Pro template generators never execute. As explained above, the template directory check at generator.js:71-77 always evaluates to true for Pro templates, preventing the dynamic generators from running. This is the single most impactful bug, rendering the entire Pro tier non-functional for all 25 Pro templates. The fix is simple: check for .hbs files specifically rather than any file, i.e., "fs.readdirSync(templateDir).some(f => f.endsWith('.hbs'))".

Bug 2: GitHub Action references non-existent dist/index.js. The action.yml specifies "main: dist/index.js" as the entry point, but no dist/ directory exists. There is no build step, no webpack/rollup configuration, and no src/action.js file that could be compiled into dist/index.js. The GitHub Action is completely non-functional and cannot work in any circumstance.

Bug 3: Fake test and lint scripts. The package.json defines "test": "echo All checks passed" and "lint": "echo Lint passed". These are no-ops that always succeed without performing any actual testing or

linting. This creates a false sense of code quality and prevents CI from catching real bugs.

Bug 4: Django template missing settings.py. The generator creates `config/__init__.py` and `manage.py` referencing `config.settings`, but never creates `config/settings.py`. The generated Django project cannot start. Similarly, the WordPress template references `includes/class-plugin.php` which is never generated, and the Spring Boot template uses the project name (which may contain hyphens) as a Java package name, which is invalid.

Bug 5: Electron template has `nodeIntegration: true`. The generated Electron app at `generator.js:1607` enables `nodeIntegration` without `contextIsolation`, which is a well-known Remote Code Execution (RCE) vulnerability. This is a textbook insecure default that security scanners flag immediately. Additionally, the Flask template uses a hardcoded fallback secret key `"dev-key"`, and the Docker Compose templates hardcode database passwords (`postgres/postgres`, `root/root`).

6. Security Vulnerability Comparison

6.1 Critical Vulnerabilities

Vulnerability	gitmoji-ai	devbadge	repokit
Trivially bypassable paywall	Yes	Yes	Yes
Hardcoded secrets in source	OAuth Client ID	No	API Key (critical)
Plaintext credential storage	Yes (GitHub PAT)	No	No
Weak hash truncation (64-bit)	Yes	Yes	N/A
Plain-text key fallback	No	Yes	No
No rate limiting	Partial	Yes	N/A
SSRF potential	No	Yes (STARSPAY_API_URL)	No
Template injection	No	No	Low (Handlebars)
Generated code vulnerabilities	No	No	Yes (Electron RCE, Flask secret, hardcoded DB passwords)

Table 8. Security vulnerability comparison across all three repositories

The most severe security issue is the hardcoded StarsPay API key in `repokit` (`sk_starspay_repokit_xK9mP2vL4nQ7rW`). This key is shipped in the npm package and could be used by attackers to interact with the StarsPay verification API directly, potentially enumerating license keys, forging verification responses, or abusing the API endpoint. The `gitmoji-ai` hardcoded OAuth Client ID (`Ov23li4G0Vn3TmY9AoFZ` at `sponsors.py:30`) is less critical because OAuth Client IDs are

semi-public, but it cannot be rotated without a code release. The plaintext storage of GitHub Personal Access Tokens in gitmoji-ai (~/.gitmoji-ai/auth.json) is also concerning, as any process on the machine can read these tokens.

6.2 Paywall Effectiveness Assessment

Across all three projects, the paywall serves as a speed bump at best. The fundamental design flaw is that license verification is performed client-side, with no server-side enforcement. An attacker with basic technical skills can bypass the paywall in under 5 minutes using any of the methods described above. The only users who would pay are those who either (a) are unaware of the trivial bypass, (b) choose to pay on principle, or (c) are non-technical users who cannot modify local files. Even for category (c), the environment variable bypass requires only a single command, making it accessible to anyone who can set an environment variable. The paywall is security theater: it exists to create the appearance of a premium product but provides no actual access control.

7. Feature Delivery Assessment

7.1 Pro Feature Delivery Rate

Project	Total Pro Features Claimed	Fully Working	Partial/Stub	Not Implemented
gitmoji-ai	7	2 (29%)	2 (29%)	3 (43%)
devbadge	7	2 (29%)	3 (43%)	2 (29%)
repokit	30	5 (17%)	10 (33%)	15 (50%)

Table 9. Pro feature delivery rate comparison

The feature delivery rates paint a clear picture: across all three projects, fewer than one-third of advertised Pro features are fully functional. The worst offender is repokit, where 50% of Pro features are completely unimplemented and only 17% work fully. When considering that the paywall is trivially bypassable, the effective value proposition of paying for Pro is near zero: users who pay are not gaining access to additional functionality that is technically protected. They are paying for features that either do not exist or are already accessible without payment.

7.2 Free Tier Value

Despite the issues with Pro features, the free tiers of all three projects do provide some value. gitmoji-ai genuinely generates quality AI commit messages and has a working git hook integration, though the free tier limits can be bypassed via the suggest command. devbadge produces valid SVG badges for 5 free types with 6 themes, though the lack of rate limiting means unauthenticated users will hit API limits quickly. repokit generates runnable projects for 5 free templates with Docker and

CI/CD configuration, which is genuinely useful for bootstrapping new projects. The free tier value is the primary reason these projects score 3-5 out of 10 rather than lower.

8. Code Quality Comparison

Metric	gitmoji-ai	devbadge	repokit
Unused dependencies	0	0	1 (ejs)
Dead code / unused imports	3	3	7+
Fake test suite	No (real tests)	No (real tests)	Yes (echo "passed")
Bug: CLI arg mismatch	No	Yes (--badges vs --badge)	No
Bug: timestamp type error	No	Yes (float - ISO string)	No
Bug: dead code path	No	No	Yes (25 generators unreachable)
Ruff lint exceptions	No	Yes (F401 suppressed)	N/A
SQL injection risk	No (parameterized)	N/A	N/A

Table 10. Code quality comparison

gitmoji-ai has the highest code quality of the three projects, with all dependencies genuinely used, parameterized SQL queries, and a real test suite. The main issues are the unused imports in `sponsors.py` and the misleading `settings.is_pro` implementation. `devbadge` suffers from the CLI argument mismatch, timestamp type error, and suppressed lint warnings. `repokit` has the worst code quality: fake tests, 7+ dead code items, an unused `ejs` dependency, and the critical dead code path that prevents Pro templates from being generated. The fake test suite in `repokit` is particularly problematic because it gives the appearance of quality assurance while catching zero bugs.

9. Overall Assessment and Scoring

Category	gitmoji-ai	devbadge	repokit
Paywall Security	1/10	1/10	1/10
Pro Feature Delivery	3/10	3/10	2/10
Free Tier Value	6/10	7/10	6/10
Code Quality	6/10	5/10	3/10

Security	4/10	4/10	2/10
Documentation Accuracy	3/10	3/10	2/10
GitHub Action Functionality	4/10	0/10	0/10
Overall	4/10	5/10	3/10

Table 11. Overall assessment scores

gitmoji-ai (4/10): The core functionality (AI commit generation) works well, and the code quality is reasonable. However, the trivially bypassable paywall, fake language support for 5 of 7 claimed languages, unimplemented Pro features (custom styles, team features), and the suggest command loophole significantly undermine the value proposition. Users who pay for Pro are getting very little additional functionality, and the paywall is effectively non-existent for anyone with basic technical knowledge.

devbadge (5/10): The free badge generation works well with 5 badge types and 6 themes. The SVG output is structurally valid and the API integration (with a token) produces accurate data. However, Pro features are mostly non-functional: animations do not work on GitHub (the primary use case), Spotify and Weather badges are placeholders, and custom colors is a dead configuration field. The broken GitHub Action and cache timestamp bug further reduce the score. The lack of rate limiting is a practical problem for users with many repositories.

repokit (3/10): This is the weakest of the three projects. The dead code bug preventing 25 Pro template generators from executing means the Pro tier delivers essentially nothing beyond what the free tier offers. The hardcoded API key is a critical security vulnerability. The fake test suite, non-functional GitHub Action, and generated code with security vulnerabilities (Electron RCE, hardcoded secrets) make this project unsuitable for production use. The free tier provides genuine value with 5 working templates, but the Pro tier is vaporware.

10. Recommendations

10.1 For Potential Users

Do not pay for Pro licenses. The paywalls in all three projects are trivially bypassable, and the Pro features are largely unimplemented or non-functional. Paying does not unlock meaningful additional functionality. The free tiers provide the genuinely useful features. For gitmoji-ai, use the free tier and manually format commits if needed. For devbadge, use the 5 free badge types and consider alternatives like Shields.io for dynamic badges. For repokit, use the 5 free templates and build additional configuration manually, or use more mature alternatives like Yeoman or create-react-app.

10.2 For the Author

If these projects are intended as legitimate open-source tools, the author should: (1) Implement server-side license verification that cannot be bypassed by modifying local files or environment

variables. (2) Remove or clearly mark as "planned" any Pro features that are not yet implemented, rather than presenting them as available. (3) Fix the critical bugs: the repokit dead code path, the devbadge GitHub Action argument mismatch, and the gitmoji-ai suggest command rate limit bypass. (4) Remove the hardcoded API key from repokit and use environment variables for all secrets. (5) Add real test suites, particularly for repokit where the current tests are no-ops. (6) Fix the generated code vulnerabilities in repokit (Electron nodeIntegration, Flask secret key, hardcoded database passwords). (7) Disclose the GitHub <style> tag stripping limitation prominently in the devbadge README when advertising animation features.

10.3 For GitHub

The public licenses.json file in the sochiautoparts/stars-pay-bot repository exposes license data for all three products. This repository should be made private or the license verification should use proper server-side validation with signed tokens (JWT) rather than client-side checks against a public file. The use of GitHub raw URLs as a license database is an abuse of the platform that undermines both GitHub users and the platform itself. Additionally, the StarsPay Telegram bot operates as an unregulated payment processor, which may warrant review under GitHub's acceptable use policies for marketplace transactions.